

FIAP

Consolidação Técnica do Cidadão Informa

HackGov para zeladoria e acessibilidade urbana

Enterprise Challenge EGESP

Gabriel Vilela - RM 562125

Luis Gustavo Rivalta - RM 561742

15 de setembro de 2026

Aplicação em produção

<https://cidadao-informa.vercel.app>

Repositório

<https://github.com/gabrielvilela12/Cidadao-Informa>

Sumário

Resumo executivo

Introdução e problema público

Parte 1 Versão Atual do Projeto

Parte 2 Ajustes no backlog

Parte 3 Implementação de API RESTful e Integração Completa

Parte 4 Manipulação Avançada de Dados e Criação de Relatórios

Parte 5 Governança e Auditoria do HackGov

Verificação técnica e implantação

Conclusão

Referências técnicas

Resumo executivo

O Cidadão Informa evoluiu para uma solução GovTech funcional que conecta a interface React, uma API Java com Spring Boot e o PostgreSQL hospedado no Supabase. O sistema registra ocorrências de zeladoria e acessibilidade urbana, organiza a triagem pública, acompanha o ciclo de atendimento e converte os protocolos em indicadores operacionais.

A versão atual apresenta CRUD completo para o recurso principal, exclusão lógica exclusiva do cidadão proprietário, relatórios estatísticos, fila persistente para jobs de IA, pilha de histórico de filtros, controle de acesso por perfil e escopo, trilha administrativa e cadeia de auditoria encadeada por hashes. A aplicação está disponível em produção e a API pública de transparência expõe dados agregados sem CPF, telefone, e-mail ou endereço residencial do cidadão.

Introdução e problema público

Ocorrências urbanas costumam chegar ao poder público por canais dispersos, com pouca padronização e baixa visibilidade sobre prioridade, localização e andamento. Esse cenário dificulta a triagem, a prestação de contas e a identificação de locais com problemas recorrentes.

O Cidadão Informa concentra esse fluxo. O cidadão registra a ocorrência, informa o local do problema e pode anexar evidências. O servidor acompanha a fila operacional, altera o status e revisa a prioridade sugerida pela IA. Gestores consultam relatórios, mapas, custos e auditorias de acordo com as permissões recebidas.

Ator	Necessidade	Resposta do sistema
Cidadão	Registrar e acompanhar uma ocorrência	Protocolo, histórico, consulta pública e exclusão lógica enquanto aberto
Servidor	Organizar a triagem e executar o atendimento	Fila, filtros, mapa, prioridade, status e custo
Gestor da prefeitura	Acompanhar equipes e indicadores	Dashboard, relatórios, permissões e exportações
Gestor da plataforma	Governar organizações e acessos globais	Onboarding, visão consolidada e auditoria administrativa

Parte 1 Versão Atual do Projeto

A referência de comparação é o resumo da Fase 4. Desde então, o projeto ampliou a arquitetura, a governança e a capacidade analítica. As mudanças foram avaliadas pelo impacto sobre o serviço público e pela evidência existente no código.

Eixo	Fase 4	Versão atual	Impacto
Arquitetura	React e funções pontuais	React, API Java em camadas, PostgreSQL e Edge Functions atrás do back-end	Centraliza regras, autenticação e contratos
Protocolos	Cadastro, consulta e mudança de status	CRUD principal completo com exclusão lógica pelo proprietário	Preserva histórico sem permitir exclusão administrativa
Gestão	Fila, dashboard e mapa básicos	Escopo regional, recorrência, permissões, custos e relatórios diários	Apoia priorização territorial e operacional
Mapa	Visualização básica das ocorrências	Visões cidadã e administrativa com filtros, geolocalização, marcadores, camada de calor e agrupamentos territoriais	Facilita a orientação do cidadão e a leitura espacial da demanda pública
Arquivos	Anexos e XLSX	Até quatro imagens JPG/PNG e exportações PDF, XLSX e CSV	Reúne evidência e prestação de contas
Dados	Indicadores básicos	Média, mediana, P90, desvio-padrão, cobertura e SLA	Qualifica decisões e evidencia limitações da amostra
Chatbot de IA	Assistente com RAG	Cache persistente por prefeitura, embeddings locais e cache do provedor	Reaproveita respostas seguras, reduz chamadas pagas e melhora a latência
Segurança	JWT e separação cidadão/admin	Escopo por perfil, prefeitura, UF e tela; CORS, rate limit e RLS	Aplica menor privilégio no servidor
Auditoria	Eventos de protocolo	Cadeia de hashes e trilha de consultas/exportações sensíveis	Permite investigar mudanças e acessos
Qualidade	Build e testes manuais	118 testes de front-end e 102 testes de back-end, lint e build	Reduz risco de regressão
Implantação	Execução local	Vercel em produção e Supabase com migrations versionadas	Permite demonstração ponta a ponta

1.1 Fluxo atual do cidadão

- Cadastro e autenticação com credenciais protegidas por hash e sessão JWT.
- Criação guiada do protocolo com categoria, descrição, endereço da ocorrência, coordenadas e imagens.
- Acompanhamento dos próprios protocolos e consulta pública pelo identificador.
- Mapa com filtros por categoria e status, navegação entre ocorrências e recurso Minha localização.
- Exclusão lógica de protocolo próprio enquanto o atendimento ainda está aberto.
- Acesso a indicadores de transparência e orientação pelo assistente virtual.

1.2 Fluxo atual da administração pública

- Fila operacional com busca, filtros e histórico para desfazer alterações.
- Mapa estratégico com marcadores, camada de calor, agrupamentos por cidade ou estado e atualização por eventos SSE.

- Revisão humana da prioridade sugerida pela IA e registro da justificativa.
- Atualização de status e custo de resolução com eventos de auditoria.
- Relatórios, exportações e indicadores respeitando o escopo do usuário autenticado.

1.3 Evolução do mapa e apoio territorial

O mapa foi redesenhado para tornar a localização dos protocolos mais clara. Na visão do cidadão, os marcadores identificam ocorrências georreferenciadas, os filtros reduzem o conjunto por categoria e status, a navegação permite avançar entre resultados e o recurso Minha localização reposiciona o mapa com autorização do navegador. Protocolos sem coordenadas confirmadas não recebem uma posição inventada.

Na visão administrativa, a equipe pode alternar entre marcadores e camada de calor, analisar concentrações por área, cidade ou estado e visualizar a chegada de novas solicitações em tempo real. Marcadores sobrepostos são distribuídos visualmente para preservar a leitura. Esses recursos apoiam a identificação de recorrências e a priorização territorial do serviço público.

A coordenada exibida representa o local da ocorrência informado ou confirmado no protocolo. Ela não deve ser interpretada como endereço residencial do cidadão. A separação reduz o risco de exposição indevida e mantém o mapa orientado ao problema público que precisa ser atendido.

Parte 2 Ajustes no backlog

O backlog mantém as histórias anteriores e acrescenta itens de front-end, segurança, arquivos, estatística, CRUD e auditoria. A prioridade considera impacto público, proteção de dados e dependência para a demonstração da solução.

ID	Tema	História resumida	Prioridade	Situação
US15	Arquivos	Anexar até quatro fotos JPG ou PNG	Alta	Implementada
US16	Transparência	Consultar indicadores e protocolo sem dados pessoais	Alta	Implementada com risco residual em fotos
US17	Segurança	Limitar servidores por estabelecimento, UF e tela	Alta	Implementada
US18	Operação	Receber solicitações e detectar recorrência territorial	Alta	Implementada
US19	Relatórios	Exportar fechamentos em PDF, XLSX e CSV	Alta	Implementada
US20	Auditoria	Registrar consulta sensível e exportação	Alta	Implementada
US21	CRUD	Excluir logicamente protocolo próprio	Alta	Implementada
US22	Arquivos	Armazenar anexos de forma privada	Alta	Parcial
US23	API	Padronizar erros e ampliar OpenAPI	Alta	Parcial
US24	Estatística	Exibir tendência, dispersão, SLA e recorrência	Média	Implementada

ID	Tema	História resumida	Prioridade	Situação
US25	Estruturas	Desfazer alterações dos filtros	Baixa	Implementada
US26	Auditoria	Pesquisar e verificar trilhas	Média	Parcial

2.1 Critérios de aceitação prioritários

US15 Fotos da solicitação

- Aceitar somente JPG e PNG e limitar o protocolo a quatro imagens.
- Validar formato, tamanho e quantidade no cliente e no servidor.
- Associar as referências persistidas ao protocolo e informar a rejeição sem apagar o formulário.

US17 Controle de acesso delegado

- Distinguir cidadão, servidor, gestor da prefeitura e gestor da plataforma.
- Validar perfil, estabelecimento, UF e permissão de tela no back-end.
- Retornar 401 sem autenticação e 403 quando faltar permissão.

US20 Auditoria administrativa

- Registrar ator, papel, ação, data, recurso, resultado e finalidade aplicável.
- Não copiar CPF, telefone, token, descrição integral ou imagem para a auditoria.
- Permitir pesquisa paginada somente a perfis autorizados.

US21 Exclusão lógica

- Permitir DELETE apenas ao cidadão proprietário e somente enquanto o protocolo estiver aberto.
- Preencher deleted_at e deleted_by sem remover fisicamente o registro.
- Ocultar o registro das consultas normais e gravar PROTOCOL_LOGICALLY_DELETED.
- Retornar 204, 401, 403, 404 ou 409 conforme o resultado.

US22 Proteção de anexos

- Usar armazenamento privado e URL assinada de curta duração.
- Evitar dados pessoais no nome físico do arquivo.
- Aplicar autorização, retenção e moderação antes de exposição pública.

US23 Contrato REST e erros

- Usar métodos HTTP e status codes coerentes.
- Manter corpo de erro previsível sem stack trace, SQL ou segredo.
- Documentar autenticação, entradas, saídas e erros relevantes no OpenAPI.

Parte 3 Implementação de API RESTful e Integração Completa

A interface React concentra as chamadas em serviços HTTP e envia o Bearer JWT. A API Java valida novamente a identidade e o escopo, executa os casos de uso e acessa o PostgreSQL. As Edge Functions de IA ficam atrás do back-end, o que impede a interface de receber credenciais administrativas.

```

React e TypeScript -> API REST Java e Spring Boot -> PostgreSQL e Supabase
|
+--> Edge Functions e serviços de IA

```

3.1 Recursos e endpoints

Método	Rota	Perfil	Finalidade	Status
POST	/api/auth/login	Público	Autentica e retorna JWT	200, 401, 429
POST	/api/auth/register	Público	Cadastra cidadão	201, 400, 409
GET	/api/auth/me	Autenticado	Retorna o perfil da sessão	200, 401
POST	/api/protocols	Cidadão	Cria protocolo e job de IA	201, 400, 401
GET	/api/protocols	Autenticado	Lista conforme o escopo	200, 401, 403
GET	/api/protocols/{id}	Autenticado	Consulta detalhe autorizado	200, 403, 404
PATCH	/api/protocols/{id}/status	Admin	Atualiza o andamento	200, 400, 403, 404
DELETE	/api/protocols/{id}	Proprietário	Executa exclusão lógica	204, 403, 404, 409
GET	/api/protocols/{id}/audit	Admin	Consulta a cadeia do protocolo	200, 403, 404
POST	/api/ai-priority/regenerate/{id}	Admin	Reprocessa a prioridade	204, 400, 403
POST	/api/ai/chat	Cidadão	Responde pelo cache ou pelo provedor de IA	200, 400, 401, 402, 429, 500
GET	/api/transparency	Público	Retorna indicadores anonimizados	200
POST	/api/admin/audit/exports	Admin	Registra exportação	201, 400, 403
GET	/api/admin/audit/events	Gestor global	Pesquisa trilha paginada	200, 400, 403

3.2 CRUD dos recursos principais

Recurso	Create	Read	Update	Delete	Situação
Protocolos	Sim	Sim	Status, prioridade e custo	Lógico pelo proprietário	Completo
Usuários	Cadastro	Perfil e consulta autorizada	Telefone e permissões	Não exposto	Parcial por política de retenção
Permissões	Sim	Sim	Substituição de escopos	Remove associação	Funcional
Relatórios	Geração	Lista e detalhe	Não aplicável	Retenção	Fechamento imutável

3.3 Integração ponta a ponta

1. O cidadão autentica-se e recebe um JWT.
2. A interface envia categoria, descrição, localização e evidências para POST /api/protocols.
3. O back-end valida os dados, identifica o cidadão pelo token e persiste o protocolo.
4. A criação gera um evento de auditoria e um job persistente de classificação por IA.

5. A API aciona a classificação imediatamente; o resultado atualiza ai_priority e ai_status.
6. O painel administrativo consulta a API, aplica o escopo do servidor e apresenta a prioridade.
7. Mudanças de status, custo ou prioridade geram novos eventos rastreáveis.

3.4 Tratamento de erros e segurança da API

A API usa status HTTP para separar validação, autenticação, autorização, ausência, conflito, limite e indisponibilidade de dependência. Mensagens entregues ao cliente não devem conter stack trace, SQL ou valores de configuração. O front-end trata carregamento, estado vazio e falha sem substituir a validação realizada no servidor.

3.5 Cache semântico e embeddings do chatbot

Antes de consumir créditos no provedor de IA, o endpoint POST /api/ai/chat procura uma resposta reutilizável no cache da prefeitura do cidadão. A busca verifica primeiro a pergunta normalizada de forma exata. Em seguida, calcula localmente um vetor de embedding com tokens, termos canônicos e trigramas e compara os candidatos por similaridade de cosseno. Essa estratégia permite reconhecer perguntas semanticamente próximas sem realizar uma chamada externa adicional para gerar o embedding.

```
pergunta -> normalização -> correspondência exata
          -> embedding local -> similaridade de cosseno
cache encontrado -> resposta sem débito na carteira
cache ausente    -> provedor de IA -> cobrança -> armazenamento seguro
```

O cache persistente é separado por establishment_id, possui validade configurável e registra quantidade e data dos acertos. Existe ainda uma segunda camada de cache no provedor para perguntas genéricas e seguras. Solicitações com CPF, e-mail, números específicos, protocolo individual, histórico anterior ou contexto pessoal não são armazenadas. Assim, a otimização reduz chamadas pagas e tempo de resposta sem compartilhar conteúdo individual entre cidadãos ou prefeituras. Como não há medição consolidada da economia percentual nesta entrega, o benefício é apresentado qualitativamente.

Parte 4 Manipulação Avançada de Dados e Criação de Relatórios

4.1 Estruturas utilizadas

Estrutura	Uso	Comportamento e motivo
Lista	Protocolos, imagens, eventos e itens de relatório	Mantém coleções ordenadas para exibição e processamento sequencial
Conjunto	UFs, permissões, papéis e identificadores	Evita duplicidade e agiliza o teste de pertencimento
Mapa	Metadados de auditoria e agrupamentos	Relaciona chaves a valores e organiza agregações
Fila FIFO	Jobs persistentes em ai_priority_jobs	Nas novas tentativas, busca created_at ASC e trata primeiro o job mais antigo
Pilha LIFO	Histórico de filtros da fila administrativa	push guarda o estado; pop recupera o último; limite de 20 snapshots

4.2 Fila FIFO de classificação por IA

Cada protocolo criado gera um registro em `ai_priority_jobs` com status `pending`. A classificação inicial é acionada imediatamente após a persistência. Se a chamada falhar ou continuar pendente, o repositório seleciona jobs com menos de três tentativas e criados nas últimas 24 horas, ordenados por `created_at` ASC. Essa ordenação materializa o comportamento FIFO nas novas tentativas.

```
Entrada: save(job)
Processamento inicial: triggerClassification(job)
Retentativa FIFO: ORDER BY j.createdAt ASC
Agendamento: fixedDelay = 300000 ms
Saída: success ou failed
```

O servidor também pode reprocessar manualmente uma classificação. O job registra status, quantidade de tentativas, resultado, mensagem de erro e datas de processamento. Como o deploy usa ambiente `serverless`, a confiabilidade das retentativas automáticas deve ser acompanhada; um cron externo dedicado é a evolução recomendada para garantir execução mesmo quando não houver instância ativa.

4.3 Pilha LIFO para desfazer filtros

A tela Fila de Solicitações utiliza `BoundedStack` para guardar os estados anteriores de busca, status, categoria, prioridade, SLA e período. Antes de uma mudança, `push` adiciona o snapshot atual ao topo. Ao selecionar Desfazer filtro, `pop` remove e devolve o último snapshot. Logo, a última alteração é a primeira a ser desfeita, conforme a regra LIFO. O limite de 20 estados evita crescimento indefinido da memória.

```
push(estadoAtual) -> adiciona ao topo
pop()             -> remove o último estado
capacidade       -> 20 snapshots
```

A ordenação visual dos protocolos mais recentes primeiro não é a pilha. Ela é apenas uma ordenação da lista operacional. A pilha está no histórico do botão Desfazer filtro.

4.4 Estatística descritiva em produção

Os números abaixo foram consultados em `GET /api/transparency` em 15 de setembro de 2026. A base possui 703 protocolos demonstrativos e não representa dados oficiais de uma prefeitura. Registros logicamente excluídos ficam fora dos indicadores normais.

Indicador	Resultado	Interpretação
Total	703 protocolos	Universo atual da base demonstrativa
Conclusão	204 ou 29%	A maior parte do estoque ainda está ativa
Backlog ativo	499	289 abertos e 210 em análise
SLA	312 atrasados; 37% no prazo ou próximo	Atrasos formam o principal grupo de intervenção
Idade do backlog	média 13,31; mediana 12,85; P90 22,55 dias	A cauda lenta passa de três semanas
Custo concluído	média R\$ 307,50; mediana R\$ 307,50; P90 R\$ 430,00	Cobertura de custo igual a 94%
Classificação de IA	703 classificados; cobertura 100%	Não havia job pendente ou com falha na extração
Tempo de resolução	amostra 0; cobertura 0%	Concluídos históricos ainda não possuem <code>resolved_at</code>

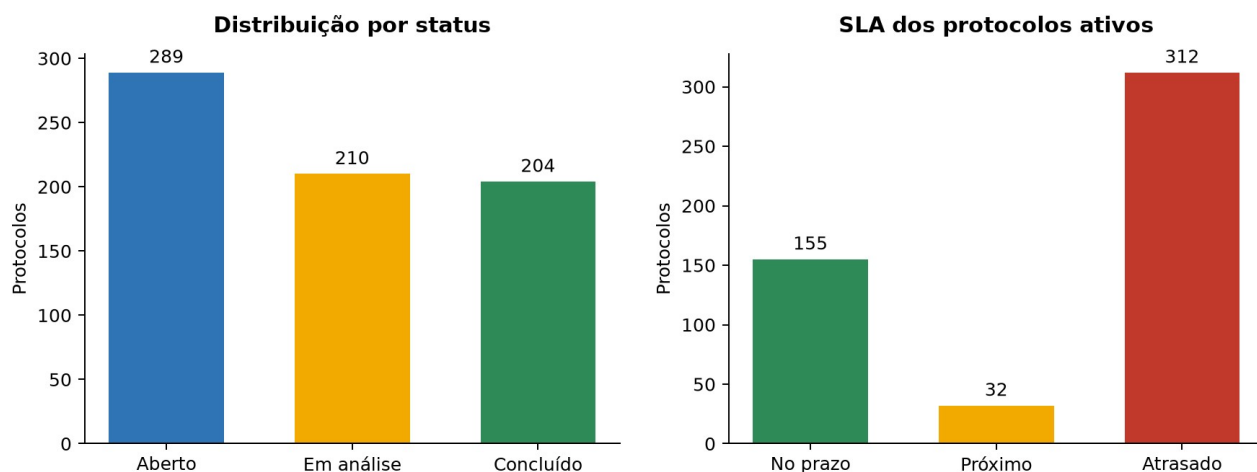


Figura 1 Distribuição por status e SLA ativo Fonte API de transparência em 15 de setembro de 2026

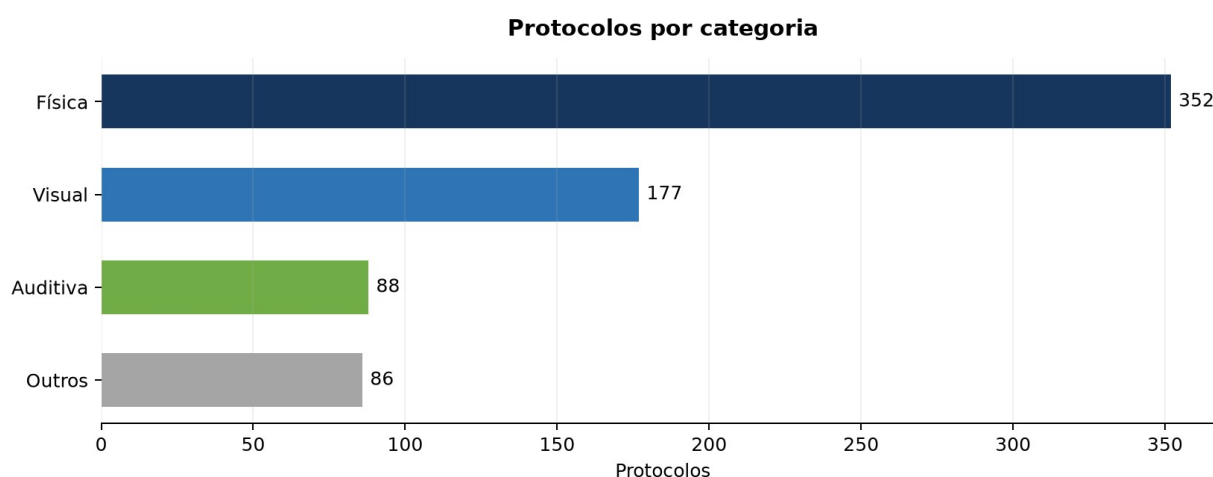


Figura 2 Distribuição por categoria Fonte API de transparência em 15 de setembro de 2026

4.5 Análise e recomendações para a gestão

- Priorizar os 312 protocolos atrasados, começando pelos classificados como críticos e altos.
- Investigar a categoria Física, que concentra 352 registros e aproximadamente metade da base.
- Usar mediana e P90 em conjunto: a mediana descreve o caso típico e o P90 evidencia a cauda lenta.
- Acompanhar a diferença mensal entre entradas e conclusões para identificar crescimento do backlog.
- Preencher resolved_at nas novas conclusões e monitorar a cobertura antes de usar tempo médio como meta.
- Usar a camada de calor e os agrupamentos territoriais para localizar concentrações de solicitações e planejar equipes de campo.
- Cruzar prioridade, atraso, recorrência e território; volume isolado não comprova maior gravidade.

As distribuições mostram concentração e associação, não causalidade. Uma região pode registrar mais protocolos por população, acesso digital ou participação social. Comparações por habitante exigem uma fonte populacional oficial e o mesmo período de análise.

Parte 5 Governança e Auditoria do HackGov

5.1 Perfis, permissões e dados pessoais

Perfil	Escopo e permissões	Acesso a dados pessoais
Cidadão	Cria, consulta e exclui logicamente protocolos próprios	Somente o próprio cadastro e seus protocolos
Servidor admin	Fila, mapa e operação dentro da prefeitura e UFs delegadas	Somente quando a tela e a função concedidas exigirem
Gestor da prefeitura	Gerencia servidores, relatórios e escopo do estabelecimento	Cadastros vinculados ao próprio estabelecimento
Gestor da plataforma	Onboarding, visão global, assinatura, consumo e auditoria	Acesso excepcional às consultas globais autorizadas
Visitante	Transparência e consulta pública	Nenhum CPF, telefone, e-mail ou endereço residencial

5.2 Operações auditadas

Operação	Evento	Dados registrados
Criação de protocolo	PROTOCOL_CREATED	Ator, status e hashes de campos sensíveis
Mudança de status	STATUS_CHANGED	Estado anterior, novo estado, ator e data
Alteração de prioridade	PRIORITY_CHANGED	Prioridade e justificativa protegida
Registro de custo	RESOLUTION_COST_UPDATED	Valor operacional e ator
Exclusão lógica	PROTOCOL_LOGICALLY_DELETED	Ator e data sem copiar descrição ou fotos
Consulta de cidadão	SENSITIVE_CITIZEN_VIEWED	Hash do recurso, finalidade e resultado
Tentativa negada	SENSITIVE_ACCESS_DENIED	Ator, tipo do recurso e resultado
Exportação	DATA_EXPORTED	Formato, quantidade, finalidade e hash opcional
Pesquisa da trilha	ADMINISTRATIVE_AUDIT_TRAIL_VIEWED	Filtros mínimos e quantidade retornada

5.3 Logs técnicos e registros de auditoria

Critério	Log técnico	Registro de auditoria
Finalidade	Diagnosticar falhas, latência e integrações	Comprovar ação relevante e responsabilização
Conteúdo	Erro, rota, tempo e identificador de correlação	Ator, ação, recurso, resultado e data
Público	Desenvolvimento e operação técnica	Governança, segurança e investigação

Critério	Log técnico	Registro de auditoria
Retenção	Pode ser curta e variar por ambiente	Segue política formal e precisa preservar integridade
Alteração	Pode sofrer rotação e agregação	Deve ser append-only na aplicação

A cadeia do protocolo armazena o hash do bloco anterior e o hash do conteúdo atual. Esse encadeamento permite detectar alterações ou remoções na sequência. A trilha administrativa complementa a cadeia ao registrar leituras e exportações que não modificam o protocolo, mas podem expor dados.

5.4 Medidas contra exposição de dados pela API

- Autenticação stateless por JWT e autorização refeita no back-end.
- Escopo por papel, estabelecimento, UF e permissão de tela.
- DTOs públicos separados dos DTOs administrativos.
- Ausência de CPF, telefone, e-mail e endereço residencial na transparência pública.
- Hash SHA-256 para identificadores sensíveis na auditoria.
- RLS e revogação de acesso direto às tabelas de auditoria para anon e authenticated.
- CORS limitado às origens configuradas e rate limiting no login.
- Segredos mantidos em variáveis de ambiente e não enviados ao navegador.
- Exclusão lógica para preservar rastreabilidade e impedir remoção administrativa indevida.
- Neutralização de fórmulas em exportações CSV e validação de formatos permitidos.

5.5 Riscos residuais e próximos controles

Risco	Situação	Próxima medida
Fotos com pessoas, placas ou residências	Moderação e armazenamento privado ainda são parciais	Bucket privado, URL assinada, remoção de metadados e política de publicação
Retentativa de IA em serverless	O agendador Spring depende de instância ativa	Criar endpoint protegido e Vercel Cron específico
Contrato de erro desigual	Há tratamento, mas nem todas as rotas usam DTO único	Padronizar código, mensagem, timestamp, rota e correlação
Histórico sem resolved_at	Cobertura atual do tempo de resolução é 0%	Acumular novas conclusões válidas sem fabricar datas antigas
Auditoria administrativa	Consulta global restrita, mas interface investigativa pode evoluir	Adicionar filtros, retenção e exportação controlada para auditor

Verificação técnica e implantação

A versão consolidada foi verificada antes da publicação. O front-end possui 118 testes automatizados; o back-end, 102. Lint e build foram concluídos. As migrations de ciclo de vida e auditoria administrativa foram aplicadas no Supabase e registradas no histórico remoto.

Verificação	Resultado
Supabase	3 colunas de ciclo de vida, 6 índices e tabela administrativa confirmados

Verificação	Resultado
RLS	Ativo; anon e authenticated sem SELECT na trilha administrativa
GitHub	Branch main sincronizada com os commits da consolidação
Vercel	Deployment de produção com status Ready
Front-end	HTTP 200 em https://cidadao-informa.vercel.app
Saúde da API	GET /api/health retornou status ok
Transparência	GET /api/transparency retornou o bloco statistics

Conclusão

O Cidadão Informa atende o objetivo de consolidar front-end, back-end, banco de dados, API REST, estruturas de dados, estatística, governança e auditoria em um único fluxo. O CRUD de protocolos está integrado à interface e preserva a responsabilidade do cidadão sobre a exclusão lógica. A fila FIFO organiza novas tentativas de classificação por IA, enquanto a pilha LIFO melhora a operação da tela administrativa.

Os relatórios tornam explícitas tanto as medidas calculadas quanto a cobertura dos dados. Os controles de acesso e as trilhas permitem identificar quem alterou, consultou ou exportou informações relevantes. O cache semântico do chatbot reutiliza respostas seguras por prefeitura e evita chamadas pagas repetidas. As próximas evoluções devem concentrar-se na proteção privada dos anexos, na padronização integral dos erros e em um cron confiável para as tentativas de IA no ambiente serverless.

Referências técnicas

- Código-fonte: <https://github.com/gabrielvilela12/Cidadao-Informa>
- Aplicação: <https://cidadao-informa.vercel.app>
- API de transparência: <https://cidadao-informa.vercel.app/api/transparency>
- Resumo da Fase 4: [entrega-fase-4/resumo-evolucao-projeto.md](#)
- Backlog: [entrega-fase-5/backlog-fase-5.md](#)
- Análise estatística: [entrega-fase-5/analise-estatistica.md](#)
- Auditoria administrativa: [entrega-fase-5/auditoria-administrativa.md](#)
- Arquitetura, cache e consumo de IA: [docs/arquitetura-assinantes-e-consumo-ia.md](#)
- Documentação de execução e endpoints: [README.md](#)